

MULTIMEDI

SPRING / SPRING BOOT TRAINING TRACK

Day 1 – Spring core, and its pitfalls

From knowing how to use Spring, to understanding why it works



DAY 1 – AGENDA

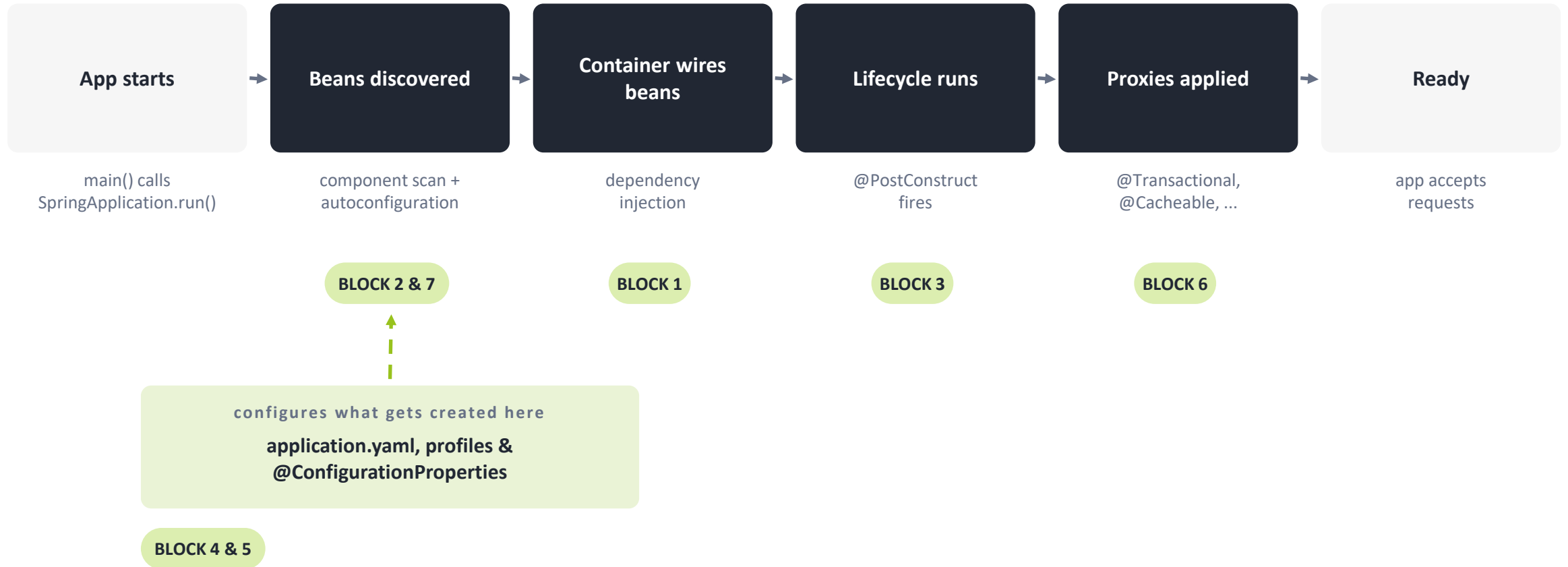
What we will cover today

01	Dependency injection & injection styles	EXERCISE
02	Application context	THEORY
03	Bean lifecycle	THEORY
04	Application.yaml & profiles	THEORY
05	@ConfigurationProperties	THEORY
06	Proxies & @Transactional	EXERCISE
07	Spring Boot autoconfiguration	EXERCISE



BEFORE WE DIVE IN

How Spring works: what happens, and when



Today's blocks, in the order Spring actually executes them



BEFORE WE DIVE IN

Spring Framework vs. Spring Boot

SPRING FRAMEWORK

You wire it yourself

- DI container, AOP, transactions
- You choose and configure your own server, data access, and modules
- Full control, more setup

SPRING BOOT

Opinionated, ready to run

- Built on top of the Spring Framework, not a replacement for it
- Autoconfiguration + starters + embedded server
- Sensible defaults, override only what you need (today's block 7)



DAY 1 • BLOCK 1

Dependency Injection & Injection Styles



A bean is just a Java object

```
new OrderService()
```

Plain Java: you create the object yourself

Spring container



a “bean”

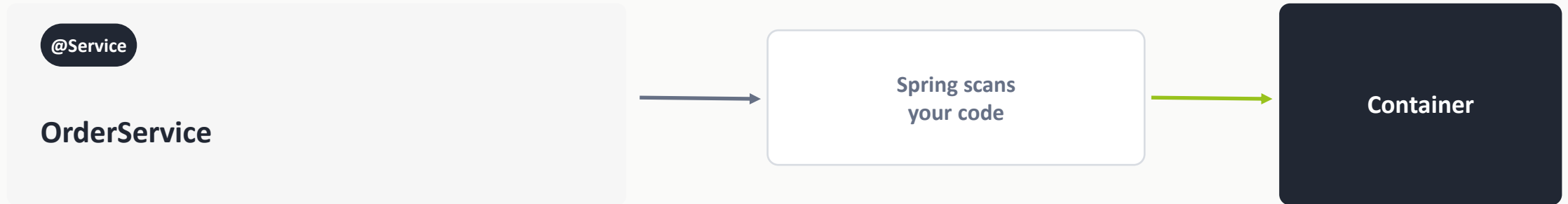
OrderService

Spring: the container creates and manages it for you

A bean is simply an object that Spring creates and holds onto, instead of you writing “new” yourself



How Spring finds your beans



A label on your class Spring looks for

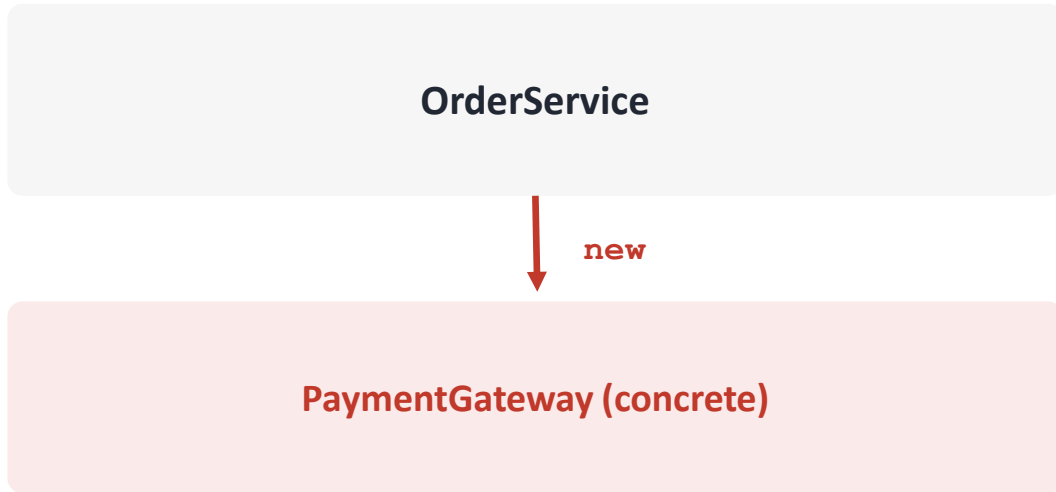
`@Component`, `@Service`, `@Repository` and `@Controller` are the labels (annotations) Spring looks for

An annotation is just a tag on your code. On its own it does nothing; Spring is the one reading it and reacting to it

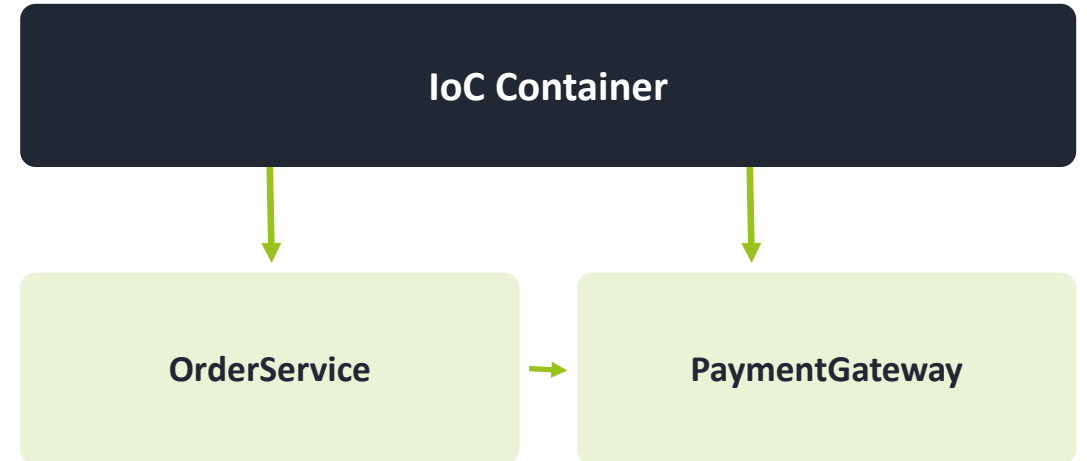


BLOCK 1 – WHAT IS IT

What is dependency injection?



Without DI: creates its own dependency



With DI: the container wires both together



Common DI pitfalls



Hidden dependencies

Field injection (@Autowired) doesn't show what a class needs just by looking at its constructor



Incomplete objects

A field-injected class can technically exist before its dependencies are actually set



Wrong bean scope

Shared mutable state where a new instance was expected, or the reverse



Field injection: quick, but risky

FIELD INJECTION

```
@Service
public class OrderService {

    @Autowired
    private PaymentGateway gateway;

    public void checkout(Order o) {
        gateway.charge(o);
    }
}
```

The field is filled in after the object already exists. Nothing in the constructor shows that OrderService actually needs a PaymentGateway.



Constructor injection: the safer default

CONSTRUCTOR INJECTION

```
@Service
public class OrderService {

    private final PaymentGateway gateway;

    public OrderService(
        PaymentGateway gateway) {
        this.gateway = gateway;
    }
}
```

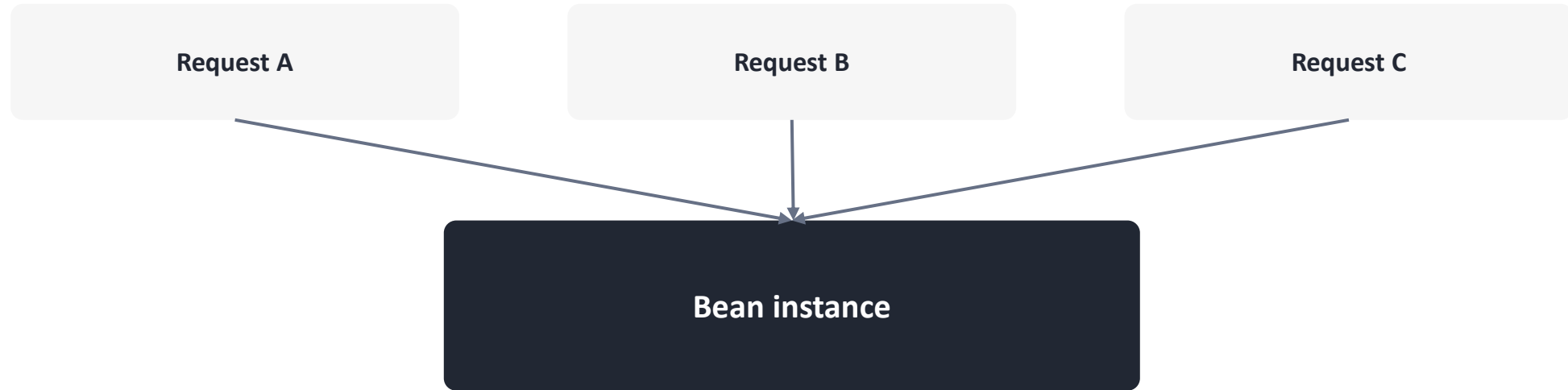
The dependency is now a constructor parameter: the object simply cannot exist without it, and it's the first thing you see when you read the class.



In your daily job: find one class in your own codebase still using field injection, and refactor it to constructor injection.



Bean scope: singleton (the default)



Every request gets the same shared instance

This is the default, and usually what you want: one instance of your service, reused everywhere. Watch out for state stored in fields here, it's shared by everyone.



BLOCK 1 – HOW TO PREVENT IT (2/2)

Bean scope: prototype



Every request gets a brand new instance

MARKING A BEAN AS PROTOTYPE

```
@Bean
@Scope("prototype")
public Basket basket() {
    return new Basket();
}
```



Part 1 – injection styles

PAYMENTGATEWAY.JAVA

```
public interface PaymentGateway {  
    void charge(double amount);  
}  
  
@Component  
public class ConsolePaymentGateway  
    implements PaymentGateway {  
    public void charge(double amount) {  
        System.out.println("Charged " + amount);  
    }  
}
```

CHECKOUTSERVICE.JAVA (STARTER)

```
@Service  
public class CheckoutService {  
  
    @Autowired  
    private PaymentGateway gateway;  
  
    public void checkout(double amount) {  
        gateway.charge(amount);  
    }  
}
```



YOUR TASK

Part 1 – injection styles

1

Start the app: CheckoutRunner calls checkout(42.0) at startup. Confirm you see “Charged 42.0” in the console (the starter uses field injection)

2

In CheckoutServiceTest, call checkout(...) on a plain “new CheckoutService()”, with no Spring involved. Note exactly what happens (and when)

3

Refactor CheckoutService to constructor injection: a private final field, set through the constructor

4

Repeat the same plain “new CheckoutService(...)” test. Compare: does it fail at the same point as before, or earlier?

Bonus: Mark the constructor parameter as `@Qualifier("consolePaymentGateway")` and add a second `PaymentGateway` implementation. What error do you get if you remove the qualifier?



Part 2 – bean scope

BASKET.JAVA (STARTER)

`@Component`

```
public class Basket {

    private final List<String> items
        = new ArrayList<>();

    public void add(String item) {
        items.add(item);
    }

    public int size() {
        return items.size();
    }
}
```

BasketA and BasketB (in the project) each get this Basket injected. Use BasketScopeTest to observe it, there is no web layer



YOUR TASK

Part 2 – bean scope

- 1 In `BasketScopeTest`: add one item through `BasketA`, then read `basket.size()` from `BasketB`. What do you see, and why?
- 2 Add `@Scope("prototype")` to `Basket` and run the exact same test again. What changed?
- 3 Write a third bean that injects `ObjectProvider<Basket>` and calls `getObject()` each time. Does this behave differently from a plain injected prototype field? Why?



DAY 1 • BLOCK 2

Application Context



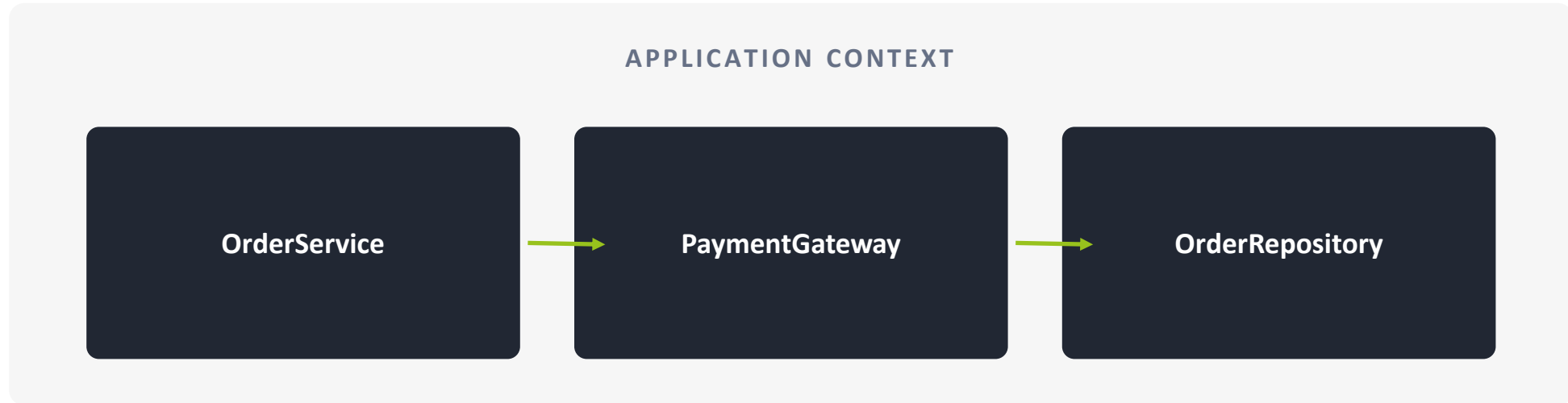
What component scanning actually does



Spring only picks up annotated classes inside the scanned package. SomeHelper has no annotation, so it's simply skipped



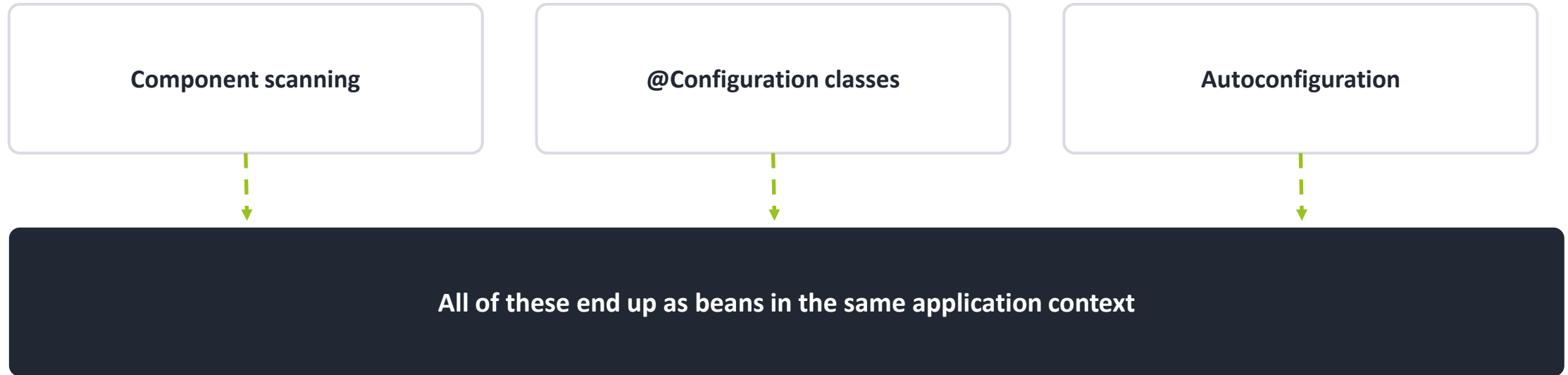
What is the application context?



The container that holds every bean, and knows how to resolve dependencies between them



Where do beans in the context come from?



Three different sources, one container. It doesn't matter to the rest of your code where a bean came from.



Bean resolution errors

THE PROBLEM

“No qualifying bean”: several beans match the same type, and Spring can't tell which one you meant

THE FIX

Mark your default with `@Primary`, or be explicit with `@Qualifier("beanName")` at the injection point

THE PROBLEM

Duplicate bean definition: two `@Bean` methods (or a scanned class and a manual one) register the same type

THE FIX

Remove the duplicate, or narrow `@ComponentScan` so the same class isn't picked up twice



In your daily job: look up one “no qualifying bean” or duplicate-bean error you've hit before, and check which of these two causes it actually was.



DAY 1 • BLOCK 3

Bean Lifecycle



Quick recap: creating an object in Java

A Spring bean is still a plain Java object underneath: its constructor still runs the same way. Spring just decides when to call it, and hands it its dependencies as arguments.

```
OrderService service  
  
= new OrderService();
```

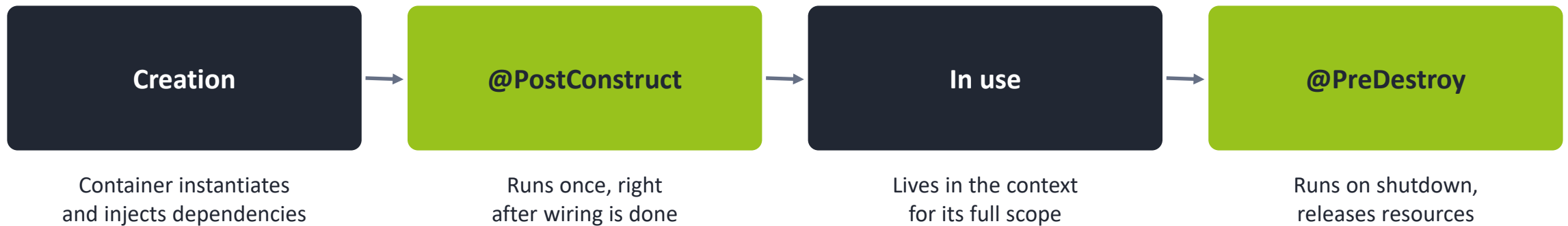


Constructor runs once, immediately

A constructor's job is normally just to set up the object's fields



What is the bean lifecycle?



Four stages every bean goes through, from the moment it's created to the moment the application shuts down



Hooking into the lifecycle

```
@PostConstruct  
public void init() {  
    // warm up cache  
}  
  
@PreDestroy  
public void shutdown() {  
    // release resources  
}
```

These two annotations are your hook into the lifecycle: one right after setup, one right before shutdown



Common lifecycle mistakes

THE PROBLEM

Heavy or dependency-sensitive logic in the constructor, before the container has finished wiring everything

THE FIX

Move that logic into `@PostConstruct`, which only runs once the bean is fully wired

THE PROBLEM

Forgetting `@PreDestroy`: connections, threads or file handles never get released on shutdown

THE FIX

Implement `@PreDestroy` (or `DisposableBean`) to release resources cleanly



In your daily job: check whether any of your `@PostConstruct`/constructor logic, or a missing `@PreDestroy`, matches what we just covered.



DAY 1 • BLOCK 4

Application.yaml & Profiles



What is YAML?

APPLICATION.YAML

```
server:  
  port: 8080  
  timeout: 30
```

SAME THING, AS .PROPERTIES

```
server.port=8080  
  
server.timeout=30
```

Same settings, two ways to write them

YAML just groups related settings by indentation (spaces), instead of repeating the same prefix on every line. Spring Boot reads this file automatically at startup.



One file, one setting per environment

APPLICATION.YAML

```
spring:
  profiles:
    active: dev

---

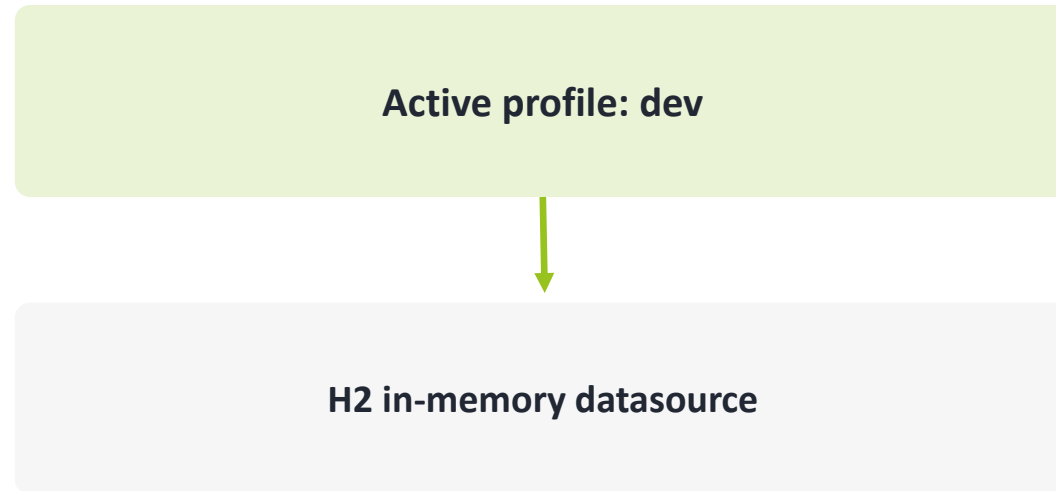
spring.config.activate.on-profile: dev
datasource.url: jdbc:h2:mem:devdb

---

spring.config.activate.on-profile: prod
datasource.url: jdbc:postgresql://prod
```



The active profile decides what runs



If “prod” were active instead, the exact same code would connect to PostgreSQL, no code change needed



Profile pitfalls

THE PROBLEM

The wrong profile, or none at all, is active: values silently fall back to defaults, sometimes in the wrong environment

THE FIX

Set the active profile explicitly at deploy time, and log or verify it on startup

THE PROBLEM

The same settings are repeated in every profile block, and drift apart over time as only some get updated

THE FIX

Keep shared config in the default (no-profile) section; override only what actually differs per profile



In your daily job: check which profile is active by default in your project, and whether any settings are needlessly repeated across profiles.



DAY 1 • BLOCK 5

@ConfigurationProperties



Quick recap: plain objects, getters and setters

A POJO (PLAIN OLD JAVA OBJECT)

```
public class MailProperties {  
    private String host;  
  
    public String getHost() {  
        return host;  
    }  
  
    public void setHost(String h) {  
        this.host = h;  
    }  
}
```



to fill it in from yaml
Spring calls setHost(...)

Private fields, reached only through public getters and setters. Spring (and most frameworks) can only fill in what it can reach.



@Value: one setting at a time

```
@Value("${mail.host}")

private String host;


@Value("${mail.port}")

private int port;


@Value("${mail.retries}")

private int retries;
```

Fine for one setting. With several related ones, it gets repetitive fast, and nothing groups them together.



@ConfigurationProperties: grouped

```
@ConfigurationProperties(  
    prefix = "mail")  
  
public class MailProperties {  
    private String host;  
    private int port;  
    private int retries;  
    // getters / setters  
}
```

One object, all related settings together, with IDE auto-completion and the ability to validate them as a group.



In your daily job: find a class with several @Value fields, and try converting it to a single @ConfigurationProperties class.

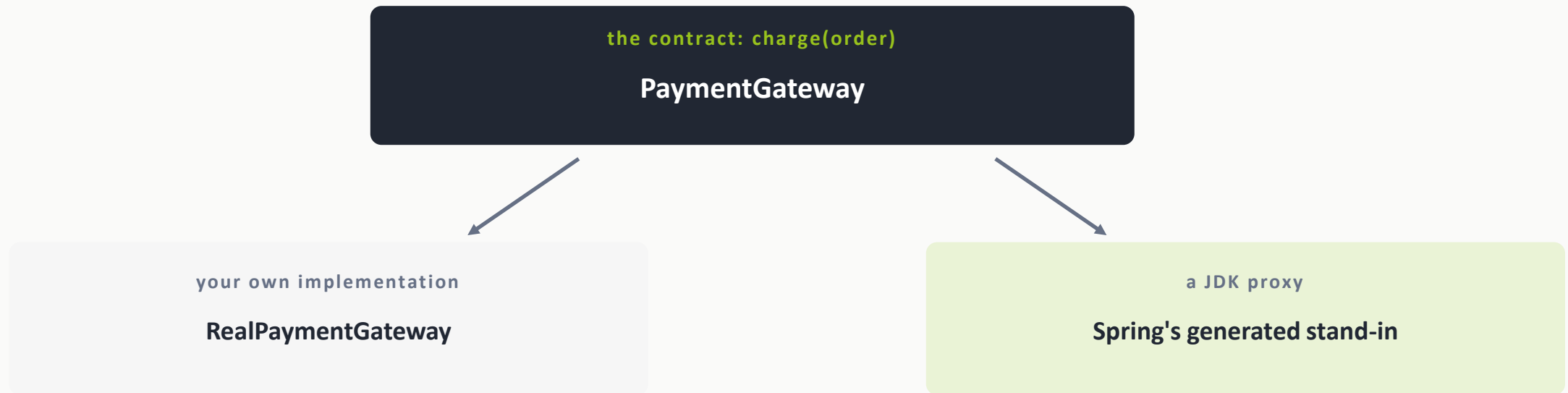


DAY 1 • BLOCK 6

Proxies & @Transactional



What is an interface?



An interface is a contract: it says which methods must exist, not how they work. Both a real class and a proxy Spring generates can implement the same one.



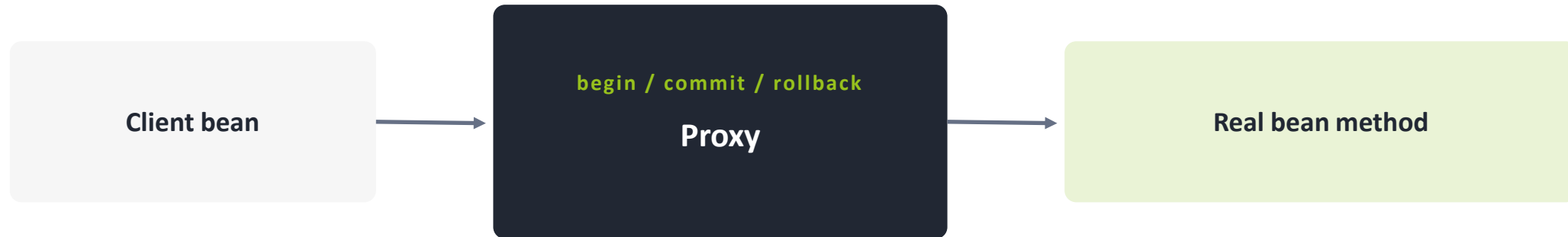
What is a database transaction?



A transaction groups operations so they succeed or fail together, data never ends up half-changed



How Spring implements @Transactional



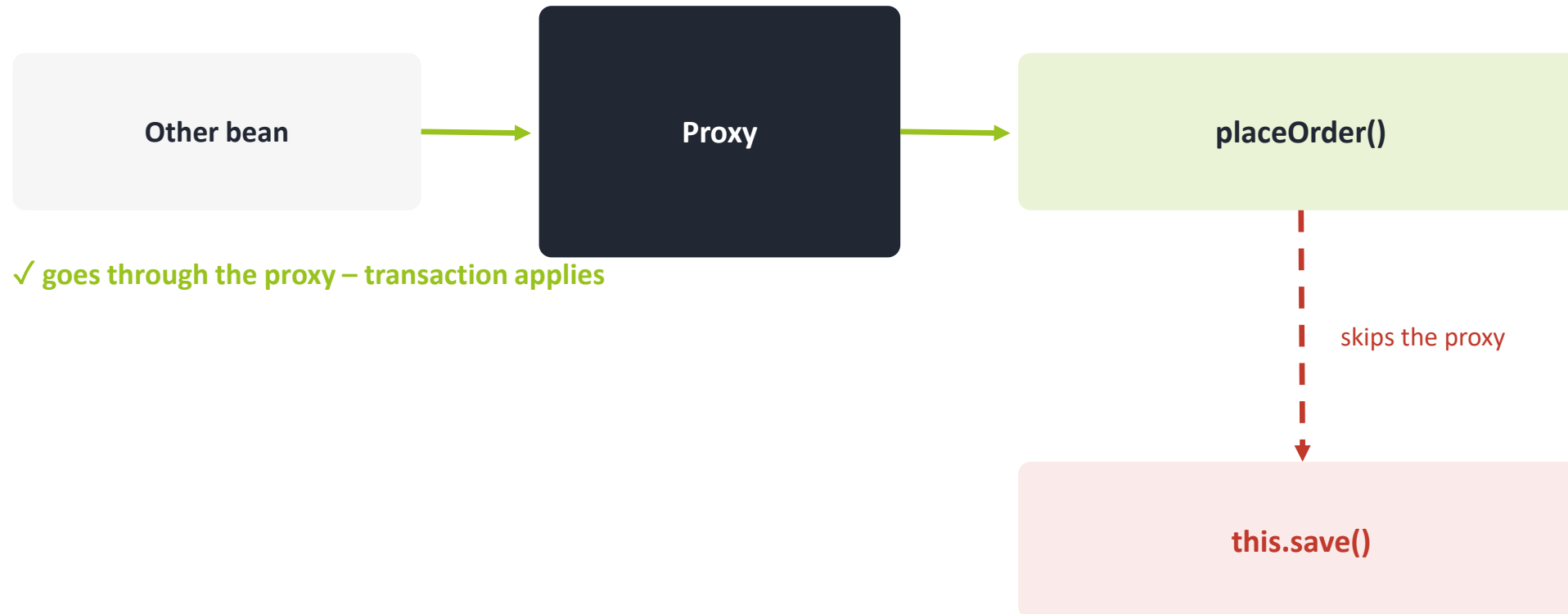
@Transactional does not touch your code directly – Spring wraps the bean in a dynamic proxy (JDK interface-based, or CGLIB subclass-based) that starts and closes the transaction around your method call

This same mechanism also powers @Cacheable and @Async



BLOCK 6 – THE PITFALL

Self-invocation silently bypasses the proxy



X same-class call – @Transactional is silently ignored



The exact same call, two different results

```
orderService.save(o);  
    // from another bean -> through the proxy -> works  
  
this.save(o);  
    // from inside the same class -> bypasses it -> silently ignored
```

Same method, same annotation. The only difference is how it was called.



The recommended fix

EXTRACT THE CALL TO A SEPARATE BEAN

Move the internally-called method into a separate bean, inject that bean, and call it from there instead of calling “this”.

The call now arrives from outside the class, so it passes through the proxy again



The fix, in code

EXTRACTED TO A SEPARATE BEAN

@Service

```
public class OrderService {  
    private final SaveHelper saveHelper;  
    public void placeOrder(Order o) {  
        saveHelper.save(o);    // external call -> through the proxy  
    }  
}
```

Last resort, messier: self-inject the proxy (@Lazy self-reference), or call via `AopContext.currentProxy()` with `exposeProxy = true`



In your daily job: search your own codebase for `@Transactional` methods called via “this”, and check whether any are silently non-transactional.



Reproducing the self-invocation bug

ORDERSERVICE.JAVA (STARTER)

```
@Service
public class OrderService {

    @Autowired
    private OrderRepository orderRepository;

    // called from a test, no @Transactional here
    public void placeOrder(Order order) {
        save(order);    // <-- self-invocation!
    }

    @Transactional
    public void save(Order order) {
        orderRepository.save(order);
        if (order.getAmount() > 1000) {
            throw new IllegalStateException("Amount too high!");
        }
    }
}
```

OrderRepository is a plain Spring Data JpaRepository<Order, Long>. Order just has an id and a double amount.



YOUR TASK

Reproducing, then fixing, the bug

1

Run `OrderServiceTest.directCall_shouldRollBack: save()` with amount > 1000 throws, and the row is NOT in the database (the transaction rolled back)

2

Look at `selfInvokedCall_currentlyLeavesTheRowBehind`. It is expected to FAIL: `placeOrder()` throws too, but is the row left in the database this time?

3

Explain to your neighbor why `placeOrder()` produces a different result than calling `save()` directly, even though `save()` does the actual work

4

Fix it: extract `save(...)` into a `SaveHelper` bean and call `saveHelper.save(order)` from `placeOrder()`. Change the direct test to `saveHelper.save(order)` too. Both tests should pass

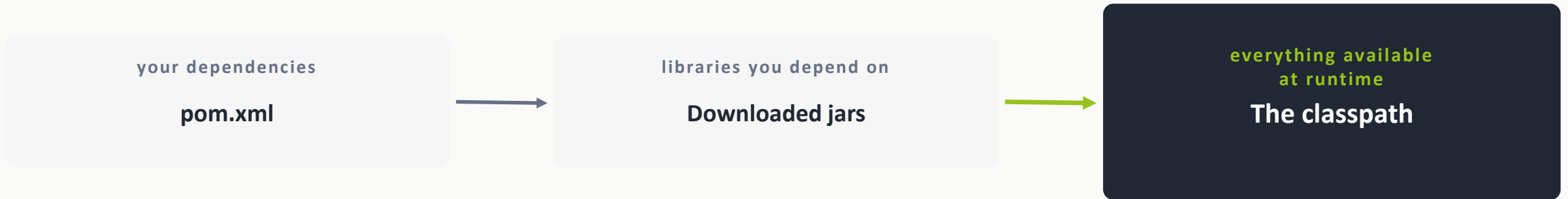


DAY 1 • BLOCK 7

Spring Boot Autoconfiguration



What is “the classpath”?

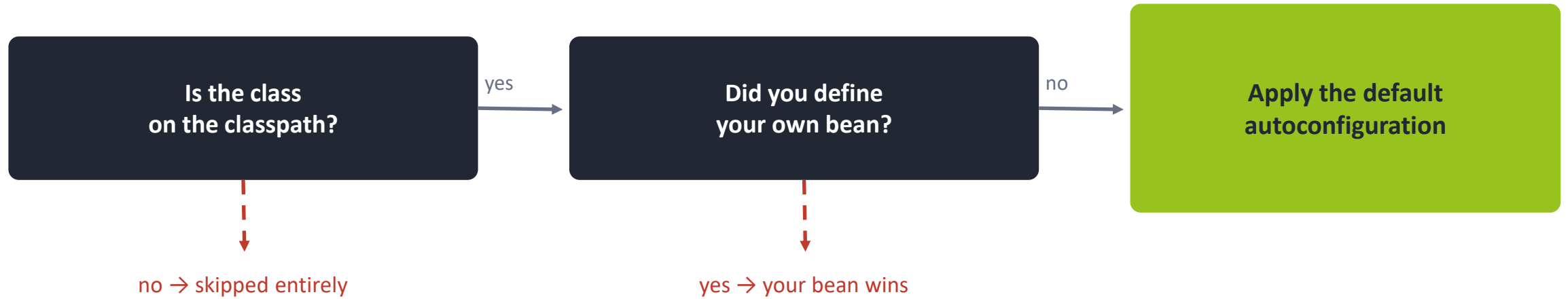


If a library was never added as a dependency, it's not on the classpath, and Spring cannot see or use it at all

“Is the class on the classpath?”, coming up next, simply means: did you add the dependency that provides it?



How Spring Boot decides what to configure





A real autoconfiguration class

```
@Configuration
@ConditionalOnClass (DataSource.class)
@ConditionalOnMissingBean (DataSource.class)
public class DataSourceAutoConfiguration {

    @Bean
    public DataSource dataSource() {
        return new HikariDataSource();
    }
}
```

This is the exact same flow from the previous slide, written as real code



When autoconfiguration surprises you



An unwanted default activates

For example, an embedded datasource you never asked for, just because the library was on the classpath



Different behavior per environment

A different classpath locally vs. in CI or production silently changes what gets configured



Conflicting starters

Two starters both try to configure the same kind of thing, and it's not obvious which one wins



Taking back control

THE PROBLEM

You don't want Boot's default for a given bean

THE FIX

Define your own bean of that type: `@ConditionalOnMissingBean` makes Boot automatically back off

THE PROBLEM

You want an autoconfiguration to never even run

THE FIX

Exclude it explicitly: `@SpringBootApplication(exclude = SomeAutoConfiguration.class)`

Not sure what happened? Run with `--debug` for the autoconfiguration report, or check the actuator `/actuator/conditions` endpoint



In your daily job: run `--debug` on one of your own projects and see what surprises you in the autoconfiguration report.



EXERCISE – STARTER CODE

A minimal starting point

APPLICATION.YAML

```
debug: true

spring:

  datasource:

    url: jdbc:h2:mem:ordersdb
```

POM.XML (RELEVANT PART)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>
    spring-boot-starter-data-jpa
  </artifactId>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
</dependency>
```

DataSourceReport (already in the project) prints the active DataSource class and JDBC URL at startup, on a line starting with [exercise3]



YOUR TASK

Reading (and changing) the autoconfiguration report

- 1 Start the app. Note the [exercise3] console line (DataSource class and JDBC URL), then search the output for “CONDITIONS EVALUATION REPORT”
- 2 Find DataSourceAutoConfiguration under “Positive matches”, and read exactly which condition(s) made it apply
- 3 Define your own DataSource bean (snippet) and restart. Which URL is used now, what happened to spring.datasource.url, and does exercise 2 still work?
- 4 Remove your bean, then add the exclude to application.yaml (snippet). Restart: what happens, and what does that tell you about exclude vs. your own bean?

Bonus: Use the exclude and your own DataSource bean at the same time. Does the app start? Why (not)?



EXERCISE – STARTER CODE

Snippet for step 3: defining your own DataSource

DATASOURCECONFIG.JAVA

@Configuration

```
public class DataSourceConfig {
```

@Bean

```
public DataSource dataSource() {  
    return new EmbeddedDatabaseBuilder()  
        .setType(EmbeddedDatabaseType.H2)  
        .build();  
}  
}
```



EXERCISE – STARTER CODE

Snippet for step 4: excluding the autoconfiguration

APPLICATION.YAML

```
debug: true

spring:

  datasource:

    url: jdbc:h2:mem:ordersdb

  jpa:

    hibernate:

      ddl-auto: update

  autoconfigure:

    exclude: org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
```

Keep autoconfigure: inside the existing spring: block. A second top-level spring: key makes the yaml invalid.



Dependency Injection & Injection Styles

- **Prefer constructor injection over field injection**

Dependencies become required constructor parameters, the field can be final, and a missing dependency fails at compile time instead of a late NullPointerException

- **Be deliberate about bean scope**

Singleton (default) is one shared instance; prototype gives a new instance per retrieval, but injecting a prototype into a singleton only resolves it once, use `ObjectProvider` for a genuinely fresh instance on every call



Application Context

- **Beans come from three sources, indistinguishably**

Component scanning, @Bean methods, and autoconfiguration all feed the same context, and nothing downstream can tell which one produced a given bean

- **Know the two resolution errors**

“No qualifying bean” means none matched; “no unique bean” means too many matched, fixed with @Primary (a default) or @Qualifier (explicit every time)



RECAP – BLOCK 3

Bean Lifecycle

- **@PostConstruct is your safe setup hook**

Runs once, immediately after the bean is fully wired, the right place for logic that depends on injected dependencies already being set

- **Don't forget @PreDestroy**

Runs once at shutdown, before the bean is discarded, missing it is how connections, threads, and file handles silently leak



Application.yaml & Profiles

- **Profiles let one codebase behave differently per environment**

The same code connects to H2 in dev and PostgreSQL in prod, purely through which profile is active, no code changes required

- **A missing or wrong profile fails silently**

The application still starts, it just falls back to default configuration, verify the active profile explicitly rather than assuming it



@ConfigurationProperties

- **Group related settings into one typed object**

@ConfigurationProperties binds a whole configuration block at once, instead of one @Value field per setting

- **It also buys you validation**

Add @Validated plus standard Bean Validation annotations, and invalid configuration fails fast at startup instead of surfacing as a mysterious null later



Proxies & @Transactional

- **@Transactional works through a proxy**

Spring wraps the bean in a proxy at creation time, which begins, commits, or rolls back the transaction around your real method call

- **Self-invocation is today's single most important pitfall**

Calling a @Transactional method via “this” bypasses the proxy entirely, silently, with no error at all; the fix is to extract that method into a separate bean and call it from there



Spring Boot Autoconfiguration

- **Autoconfiguration follows one simple, inspectable rule**

It applies only if the relevant class is on the classpath, and only if you haven't already defined your own bean of that type

- **You can always see exactly what happened, and take control**

Run with `--debug` (or check `/actuator/conditions`) to see every autoconfiguration considered; exclude one explicitly if you want it never even considered, regardless of your own beans



LOOKING AHEAD

Day 2: Spring Data JPA & Messaging

Persistence context & transactions in practice – data consistency – asynchronous integration – messaging fundamentals



WRAP-UP

In your daily job – check these in your own projects



Block 1

Refactor one class still using field injection to constructor injection



Block 2

Look up a past “no qualifying bean” or duplicate-bean error you've hit



Block 3

Check your `@PostConstruct`/constructor logic, and any missing `@PreDestroy`



Block 4

Check which profile is active by default, and for settings repeated across profiles



Block 5

Find a class with several `@Value` fields and convert it to `@ConfigurationProperties`



Block 6

Search for `@Transactional` methods called via “this” that might be silently ignored



Block 7

Run `--debug` on one of your own projects and see what the autoconfiguration report shows



DON'T FORGET

Quiz

15-question recap quiz – Day 1, startup flow through autoconfiguration

Questions?

Thank you for today – see you on Day 2

Your feedback

Scan the code below to fill in the feedback form

